ALScreen Framework Pseudocode

ScoreFormer.Train

Input: $P = \{(m_i, y_i)\}$: training pool, molecules m_i and their docking scores y_i;
H: hyperparameters (optimizer, learning-rate schedule, batch size,
weighting strength α, log-weight cap c, early-stopping patience p);
E: maximum number of training epochs.

Output: f_{theta}: trained ScoreFormer.

$G \leftarrow \{(G_i, y_i)\}$, with $G_i = \text{Graph}(m_i)$, for each (m_i, y_i) in P.
Split G into a training set G_{tr} and a validation set G_{val}.
Initialize θ, the optimizer, and the learning-rate schedule from H.
$l^* \leftarrow +\infty$; $\theta^* \leftarrow \theta$; $s \leftarrow 0$.
for e = 1, …, E **do**
    Set f_{theta} to training mode.
    for each mini-batch $B \subseteq G_{tr}$ **do**
        $\hat{y}_i \leftarrow f_{theta}(G_i)$ for each $i \in B$.
        $w_i \leftarrow \exp(\min(-\alpha \cdot y_i, c))$ for each $i \in B$. // more negative (better) scores get larger weights
        $L_B \leftarrow (1 / |B|) \cdot \sum_{i \in B} w_i \cdot (\hat{y}_i - y_i)^2$.
        Update θ using the gradient of L_B and the optimizer.
    end for
    Set f_{theta} to evaluation mode.
    Compute validation loss and prediction metrics on G_{val}. // monitoring only
    $\theta^{(e)} \leftarrow \text{copy}(\theta)$; save checkpoint $\theta^{(e)}$.
    Step the learning-rate schedule.
end for
return $f_{theta}^{(E)}$.

ScoreFormer.Infer

Input: $Q = \{m_i\}$: molecules m_i to be scored;
$f_{theta}^{(E)}$: trained model returned by ScoreFormer.Train.
Output: $\hat{Y} = \{(m_i, \hat{y}_i)\}$: predicted docking scores keyed by molecule identifier.

Set $f_{theta}^{(E)}$ to evaluation mode (dropout off, gradients disabled).
$\hat{Y} \leftarrow \varnothing$.
for each mini-batch $B \subseteq Q$ **do**
    $G_B \leftarrow \{\text{Graph}(m_i) : m_i \text{ in } B\}$.
    $\hat{y}_B \leftarrow f_{theta}^{(E)}(G_B)$.
    $\hat{Y} \leftarrow \hat{Y} \cup \{(m_i, \hat{y}_i) \text{ for } m_i \text{ in } B\}$.
end for
return $\hat{Y}$.

ALScreen (active-learning virtual screening)

Input: L: molecular library;
O_T: docking oracle for target T;
n_{sub}: initial subset size;
n_{oracle}: molecules docked per cycle;
n_{pool}: molecules added to the training pool per cycle;
K: maximum number of cycles;
H, E: training settings;

Output: R: ranked list of prioritized molecules.

$S \leftarrow \text{RepresentativeSubset}(L, n_{sub})$.
$D_0 \leftarrow O_T(S)$. // initial training pool $\{(m_i, y_i)\}$
$P_0 \leftarrow D_0$; $D_{all} \leftarrow D_0$. // training pool; all docked molecules
$f^{(0)} \leftarrow \text{ScoreFormer.Train}(P_0, H, E)$. // initial training
for $k = 1, \ldots, K$ **do**
  $\hat{Y}_k \leftarrow \text{ScoreFormer.Infer}(L, f^{(k-1)})$.
  $G_k \leftarrow$ the n_{oracle} molecules in L with the lowest predicted scores in $\hat{Y}_k$.
  $D_k \leftarrow O_T(G_k)$.
  $D_{all} \leftarrow D_{all} \cup D_k$.
  $\{e_i\} \leftarrow \{|\hat{y}_i - y_i|\}$ for each (m_i) in D_k.
  $C_k \leftarrow$ the n_{pool} best- and n_{pool} worst-predicted molecules of D_k (lowest and highest e_i), with their docking labels.
  $P_k \leftarrow P_{k-1} \cup C_k$.
  $f^{(k)} \leftarrow \text{ScoreFormer.Train}(P_k, H, E)$. // retrained from scratch
end for
$\hat{Y}_{K+1} \leftarrow \text{ScoreFormer.Infer}(L, f^{(K)})$.
$G_{K+1} \leftarrow$ the n_{oracle} compounds in L with the lowest predicted scores in $\hat{Y}_{K+1}$.
$D_{K+1} \leftarrow O_T(G_{final})$. // final docking round
$D_{all} \leftarrow D_{all} \cup D_{K+1}$.
$R \leftarrow D_{all}$ sorted by ascending docking score y_i.
return R.